Dokumentasi Teknis Aplikasi — IMA-ISA (Media Intelligence & AI Clipping Studio)

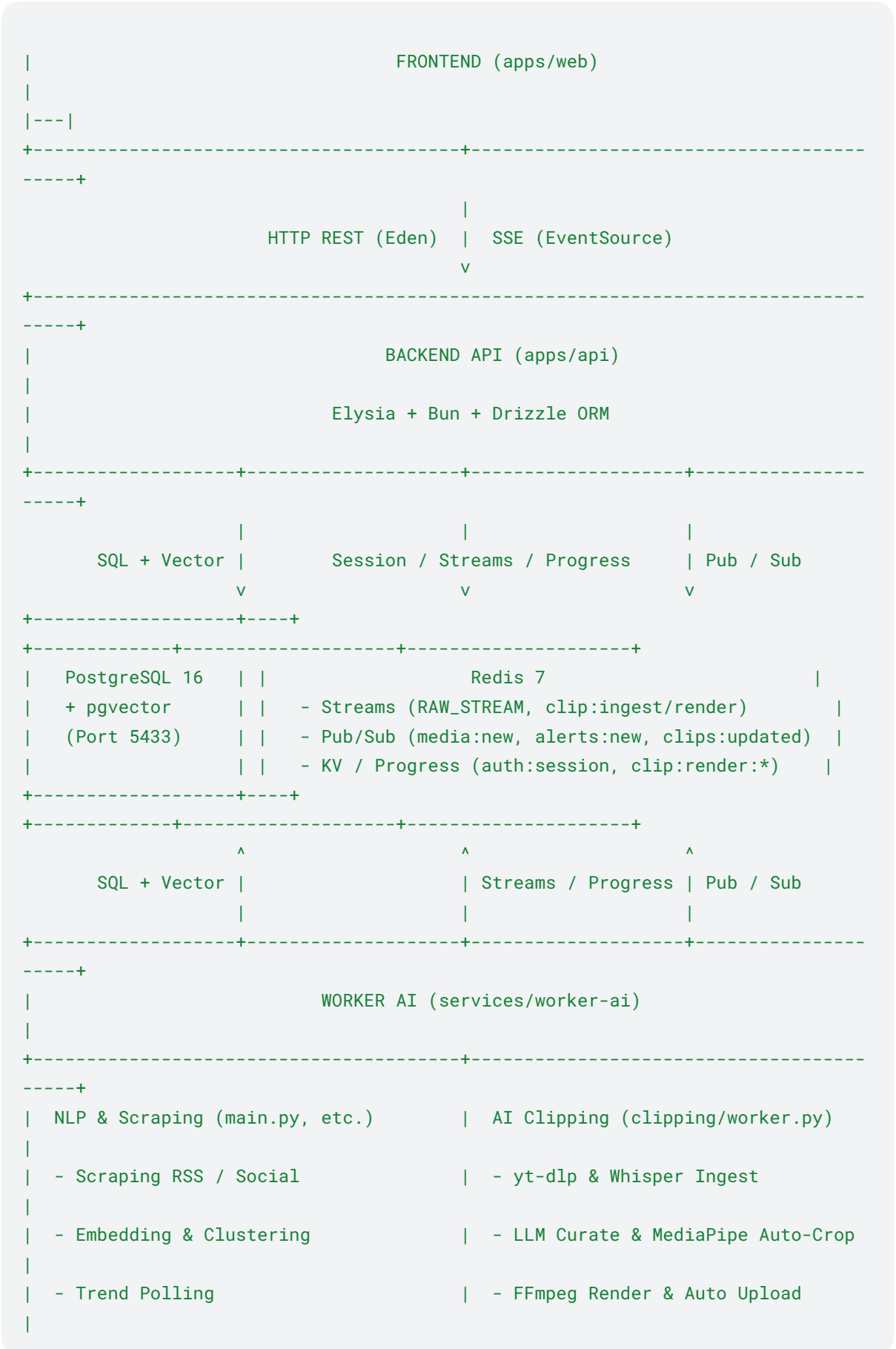
Dokumen ini merangkum arsitektur, seluruh library yang digunakan, skema database & Redis, pemetaan lengkap relasi file Frontend–Backend–Worker, daftar endpoint API, konfigurasi environment, dan perintah operasional aplikasi IMA-ISA.

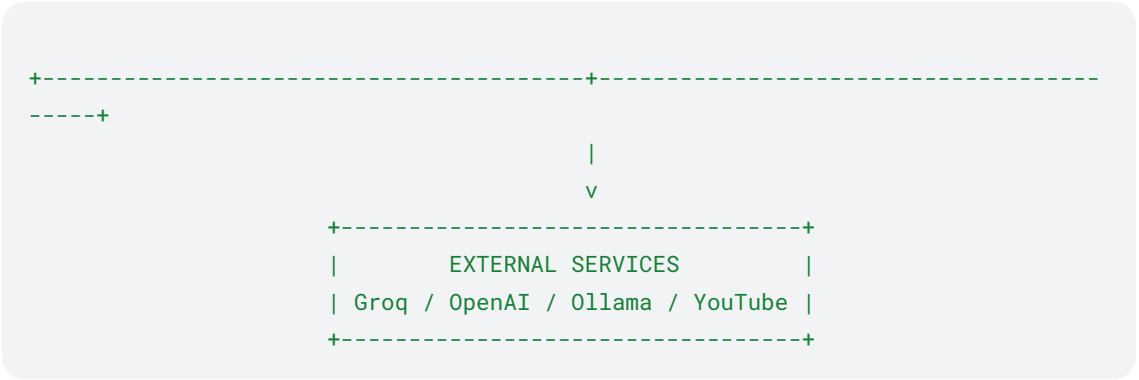
1. Ringkasan Arsitektur

IMA-ISA dirancang sebagai platform terintegrasi menggunakan arsitektur **monorepo TypeScript (Bun workspace)** yang digabungkan dengan **Python worker** terisolasi. Sistem ini terbagi menjadi empat lapisan utama:

1. **Frontend (`apps/web`)**: Single-Page Application (SPA) berbasis React 19, Vite, Tailwind CSS, dan TanStack Query. Terhubung secara terstruktur ke backend menggunakan Eden Treaty (`@elysiajs/eden`) untuk HTTP REST terdistribusi serta `EventSource` (SSE) untuk komunikasi waktu-nyata (realtime updates).
2. **Backend API (`apps/api`)**: Server web berkinerja tinggi berbasis Elysia yang berjalan di atas runtime Bun. Bertanggung jawab mengelola autentikasi berbasis session, menglayani REST API, menyiarkan streaming SSE (melalui Redis Pub/Sub bridge dan polling progress render), serta mengeksekusi query database ke PostgreSQL melalui Drizzle ORM.
3. **Shared Types (`packages/types`)**: Workspace internal murni TypeScript yang mendefinisikan kontrak tipe data bersama (domain interfaces, skema DTO, serta utilitas tipe). Impor dilakukan langsung oleh `apps/api` dan `apps/web` untuk menjamin ketatnya *end-to-end type safety*.
4. **AI & Media Worker (`services/worker-ai`)**: Layanan Python independen yang mengelola pemrosesan berat asynchronous. Terbagi menjadi dua subdomain utama:
    - **NLP & Scraping**: Ingest/scraping berita, RSS, dan media sosial; ekstraksi NLP (vektor embedding 768-dimensi, Named Entity Recognition, analisis sentimen, perangkuman/summarization); clustering topik otomatis; serta *trend polling*.
    - **AI Video Clipping Studio**: Pipeline pemrosesan media komprehensif mencakup ingest video (`yt-dlp`), transkripsi suara (`faster-whisper` / `mlx-whisper`), kurasi segmen berbasis LLM, deteksi visual & auto-framing (`MediaPipe` & `OpenCV`), pembuatan subtitle sintetis, rendering (`FFmpeg`), serta otomasi distribusi & pelacakan performa (`Playwright` & `uiautomator2`).

```
None
+-------------------------------------------------------------------------
-----+
```


FRONTEND (apps/web)
HTTP REST (Eden)
SSE (EventSource)
BACKEND API (apps/api)
Elysia + Bun + Drizzle ORM
SQL + Vector
Session / Streams / Progress
Pub / Sub
PostgreSQL 16
+ pgvector
(Port 5433)
Redis 7
- Streams (RAW_STREAM, clip:ingest/render)
- Pub/Sub (media:new, alerts:new, clips:updated)
- KV / Progress (auth:session, clip:render:*)
SQL + Vector
Streams / Progress
Pub / Sub
WORKER AI (services/worker-ai)
NLP & Scraping (main.py, etc.)
- Scraping RSS / Social
- Embedding & Clustering
- Trend Polling
AI Clipping (clipping/worker.py)
- yt-dlp & Whisper Ingest
- LLM Curate & MediaPipe Auto-Crop
- FFmpeg Render & Auto Upload



2. Tech Stack & Daftar Lengkap Library

2.1 Runtime & Monorepo Root (`package.json`)

Library / Tool	Versi	Fungsi & Peran Utama
`bun`	`>=1.2.0`	Runtime JavaScript/TypeScript utama dan package manager untuk Bun workspace.
`concurrently`	`^9.1.2`	Menjalankan seluruh proses backend, frontend, dan Python worker secara paralel pada mode pengembang.
`typescript`	`^5.9.3`	System compiler dan type-checker terpadu di seluruh modul monorepo.

2.2 Frontend (`apps/web/package.json`)

Package	Versi	Fungsi & Peran Utama
`react`, `react-dom`	`^19.3.1`	Core UI library dan DOM renderer berbasis React 19.
`react-router-dom`	`^7.18.5`	Management routing aplikasi client-side (`/`, `/login`, `/media`, `/clusters`, `/trending`, `/clipping`, `/alerts`, `/settings`).

Package	Versi	Fungsi & Peran Utama
`@tanstack/react-query`	`^5.107.1`	Asynchronous state management, caching, background refetching, dan auto invalidation via SSE events.
`@elysiajs/eden`	`^1.4.9`	Client HTTP type-safe yang terintegrasi secara langsung dengan tipe backend Elysia.
`lucide-react`	`^0.468.0`	Library ikonografi UI.
`recharts`	`^2.15.4`	Visualisasi data analitis (grafik volume platform, tren sentimen, *share of voice*).
`maplibre-gl`	`^6.17.0`	Peta interaktif berbasis GPU untuk sebaran lokasi geografis media/isu.
`react-simple-maps`	`^3.0.0`	Rendering peta kustom berbasis data SVG.
`world-atlas`	`^2.0.2`	Dataset prapemrosesan topografi geografi dunia berbasis TopoJSON.
`@radix-ui/react-dialog`	`^1.1.15`	Komponen modal dialog yang memenuhi standar aksesibilitas WAI-ARIA.
`@radix-ui/react-slot`	`^1.2.3`	Utility komposisi elemen komponen UI tanpa menambahkan wrapper node DOM.
`class-variance-authority`	`^0.7.1`	Mengelola varian styling CSS secara declarative.
`clsx`, `tailwind-merge`	`^2.1.0`, `^2.6.0`	Penggabungan kelas CSS Tailwind dan resolusi bentrokan (*class conflict*).
`@media-intel/types`	`workspace:*`	Package internal tipe domain TypeScript.
DevDependencies		

Package	Versi	Fungsi & Peran Utama
`vite`, `@vitejs/plugin-react`	`^7.3.8`, `^5.1.0`	Fast bundler & HMR server untuk React.
`tailwindcss`, `postcss`, `autoprefixer`	`^3.4.19`, `^8.5.6`, `^10.4.21`	Utility-first CSS framework dan processor pipeline.
`@types/*` (`react`, `dom`, `bun`, `node`, `geojson`)	Various	Definisi tipe untuk modul pengembang frontend.

2.3 Backend API (`apps/api/package.json`)

Package	Versi	Fungsi & Peran Utama
`elysia`	`^1.4.19`	Framework HTTP server berbasis Bun dengan dukungan TypeBox dan High-Performance Route Handler.
`@elysiajs/cors`	`^1.4.3`	Middleware Cross-Origin Resource Sharing untuk API dan SSE stream.
`@elysiajs/swagger`	`^1.3.1`	Pembuat dokumentasi OpenAPI (Swagger UI) otomatis pada endpoint `/docs`.
`drizzle-orm`	`^0.45.4`	Object-Relational Mapper type-safe yang mendukung ekstensi `pgvector`.
`postgres`	`^3.4.7`	Driver native PostgreSQL berkinerja tinggi untuk Bun runtime.
`ioredis`	`^5.6.1`	Redis client yang menangani Pub/Sub, Streams, Session Store, dan Progress Tracking.
`@media-intel/types`	`workspace:*`	Package internal tipe domain TypeScript.
DevDependencies		

Package	Versi	Fungsi & Peran Utama
`drizzle-kit`	`^0.31.11`	CLI tool untuk skema migration generation dan database introspection.

2.4 Shared Package (`packages/types/package.json`)

Package	Versi	Fungsi & Peran Utama
`@media-intel/types`	`workspace:*`	Murni modul definisi tipe TypeScript (`^5.9.3`), tanpa runtime dependency eksternal.

2.5 Python Worker (`services/worker-ai`)

Library / Requirement File	Versi Minimum	Fungsi & Peran Utama
Scraping & DB (`requirements.txt`)		
`feedparser`	`6.0.11`	Parsing feed RSS / Atom dari berbagai sumber media berita.
`trafilatura`	`2.0.0`	Ekstraksi konten teks bersih dan metadata dari halaman web HTML.
`beautifulsoup4`, `lxml`	`4.12.3`, `5.3.0`	Web scraping fallback dan parsing dokumen XML/HTML.
`requests`	`2.32.3`	Client HTTP synchronous untuk panggilan API eksternal.
`psycopg[binary]`	`3.2.3`	Native PostgreSQL driver 3 untuk Python dengan konversi array `pgvector`.
`redis`	`5.2.1`	Redis Python client untuk konsumsi Stream dan komand Pub/Sub.
`python-dotenv`, `numpy`	`1.0.1`, `>=1.26`	Manajemen environment dan manipulasi larik numerik vektor.

Library / Requirement File	Versi Minimum	Fungsi & Peran Utama
AI / NLP (`requirements-ai.txt`)		
`torch`	`>=2.4, <3`	PyTorch framework untuk inference model deep learning.
`transformers`	`>=4.46, <5`	Pustaka HuggingFace untuk eksekusi model NLP, NER, dan summarization.
`sentencepiece`, `spacy`	`>=0.2`, `>=3.8`	Tokenizer dan pemrosesan Bahasa Alami (*Linguistic Pipeline*).
Video Clipping (`requirements-clipping.txt`)		
`yt-dlp`	`>=2024.12.13`	Pengunduh video/audio multi-platform teroptimasi.
`faster-whisper` / `mlx-whisper`	`>=1.1.0` / `>=0.4`	Transkripsi suara ke teks berbasis Whisper (Inference CTranslate2 / Apple MLX).
`av`, `librosa`, `soundfile`	`>=15`, `>=0.10.2`	Pemrosesan sinyal audio, pemotongan, dan ekstraksi spektrum.
`pillow`, `opencv-python-headless`	`>=10.4`, `>=4.10`	Pemrosesan citra frame video dan manipulasi matriks piksel.
`mediapipe`	`>=0.10.18`	Framework ML untuk Face Detection dan Auto-framing rasio 9:16.
`playwright`, `uiautomator2`	`>=1.49`, `>=3.2`	Otomasi browser web & perangkat Android untuk publikasi konten.

2.6 Infrastruktur & System Binaries (`docker-compose.yml` & `deploy/`)

- **PostgreSQL + pgvector**: Base image `pgvector/pgvector:pg16` (Host Port: `5433` → Container Port: `5432`).
- **Redis**: Base image `redis:7-alpine` (Host Port: `6379`), dikonfigurasi dengan mode AOF (*Append-Only File*).

- **FFmpeg / FFprobe**: System binary lokal/container untuk pemotongan, pemrosesan filter, merging, dan encode video/audio.
- **Nginx**: Web server reverse-proxy untuk SSL termination dan routing HTTP/SSE di lingkungan produksi.

3. Database PostgreSQL, Migrasi & Redis

3.1 Tabel PostgreSQL

Seluruh tabel dikelola menggunakan Drizzle ORM. Skema terbagi menjadi dua file utama:

Modul Media Intelligence (`apps/api/src/db/schema.ts`)

Tabel	Primary Key	Fungsi Utama & Kolom Kunci
`sources`	`id (UUID)`	Menyimpan direktori pendaftaran media berita/kanal sosial. Kolom: `name`, `url`, `type` (RSS, Twitter, YouTube), `fetch_interval_minutes`.
`media_items`	`id (UUID)`	Item artikel/berita yang di-ingest. Kolom: `title`, `content`, `url`, `published_at`, `sentiment` (positive, neutral, negative), `embedding` (`vector(768)`).
`topic_clusters`	`id (UUID)`	Pengelompokan topik secara spasial/vektor. Kolom: `title`, `summary`, `category`, `article_count`, `centroid` (`vector(768)`).
`cluster_items`	`(cluster_id, media_item…`	Tabel pendaftaran *many-to-many* antara kluster dan item berita.
`monitored_keywords`	`id (UUID)`	Kata kunci pantauan sistem. Kolom: `keyword`, `category`, `is_active`.
`social_trends`	`id (UUID)`	Data tren terkini platform. Kolom: `platform` (X, TikTok, Google), `trend_name`, `volume`, `sentiment_score`.

Tabel	Primary Key	Fungsi Utama & Kolom Kunci
`alerts`	`id (UUID)`	Log peringatan dini sistem. Kolom: `title`, `severity` (info, warning, critical), `cluster_id`, `status` (new, acknowledged).

Modul AI Video Clipping (`apps/api/src/db/schema/clipping.ts`)

Tabel	Primary Key	Fungsi Utama & Kolom Kunci
`clip_sources`	`id (UUID)`	Sumber media input video. Kolom: `url`, `title`, `duration`, `status` (pending, ingested, failed), `metadata` (JSONB).
`generated_clips`	`id (UUID)`	Segmen klip video hasil pemrosesan AI. Kolom: `source_id`, `start_time`, `end_time`, `virality_score`, `rendering_status`, `video_path`.
`clip_distributions`	`id (UUID)`	Tracking publikasi klip ke media sosial. Kolom: `clip_id`, `platform`, `post_url`, `status` (scheduled, published, failed), `metrics` (JSONB).

Catatan Autentikasi: Pengelolaan identitas penggunanya tidak membutuhkan tabel `users` khusus pada PostgreSQL. Sistem menggunakan kredensial lingkungan (`AUTH_USERNAME`, `AUTH_PASSWORD`, `AUTH_PASSWORD_HASH`) serta menyimpan token session aktif secara terdistribusi di Redis (auth:session:<token> dengan waktu kedaluwarsa TTL 7 hari), serta didukung fallback store in-memory `Map` pada `apps/api/src/lib/auth.ts`.

3.2 File Migrasi Drizzle (`apps/api/drizzle/`)

Proses eksekusi migrasi ditangani oleh file runner `apps/api/src/db/migrate.ts`.

File Migrasi SQL	Deskripsi Perubahan Skema
`0000_ambitious_red_shift.sql`	Inisialisasi tabel dasar `sources`, `media_items`, dan indeks pgvector awal.

File Migrasi SQL	Deskripsi Perubahan Skema
`0001_spicy_shadowcat.sql`	Penambahan tabel `topic_clusters` dan `cluster_items`.
`0002_mysterious_spitfire.sql`	Penambahan tabel `monitored_keywords` dan penyesuaian tipe enum.
`0003_slow_hulk.sql`	Penambahan skema tabel `social_trends`.
`0004_stiff_mockingbird.sql`	Penambahan tabel `alerts` untuk pengiriman notifikasi terotomasi.
`0005_overjoyed_ghost_rider.sql`	Pembaruan struktur indeks pencarian HNSW vektor pada `media_items`.
`0006_yielding_iron_lad.sql`	Penyesuaian kolom JSON metadata pada `media_items` dan `sources`.
`0007_stiff_black_widow.sql`	Restrukturisasi indeks gabungan `cluster_items`.
`0008_clipping.sql`	Inisialisasi penuh tabel pemrosesan klip video (`clip_sources`, `generated_clips`, `clip_distributions`).

3.3 Redis Channels, Streams & Keys

Redis Pub/Sub Channels

Channel	Penerbit (Publisher)	Pelanggan (Subscriber)	Fungsi / Tujuan Utama
`media:new`	Worker NLP / Scraper	API Server → SSE	Penyiaran notifikasi artikel berita baru yang berhasil di-ingest.
`alerts:new`	Worker NLP / Alerts	API Server → SSE	Penyiaran sinyal indikator krisis atau peringatan dini ke UI.
`clusters:updated`	Cluster Worker	API Server → SSE	Menginformasikan pembaharuan struktur kluster topik terkini.
`clips:updated`	Worker AI Clipping	API Server → SSE	Notifikasi siklus hidup klip (status transkripsi, render, publikasi).

Redis Streams

Stream Key	Produsen (Producer)	Konsumen (Consumer)	Fungsi Utama
`RAW_STREAM` / `media:raw`	Scraper Worker	NLP Embedding Worker	Antrean item berita mentah yang menunggu generasi vektor dan ekstraksi NLP.
`clip:ingest`	API Server	Clipping Worker	Antrean pengunduhan video (`yt-dlp`) dan proses ekstraksi transkrip.
`clip:render`	API Server / Worker	Clipping Worker	Antrean instruksi rendering potongan klip (`FFmpeg`, `MediaPipe`).
`clip:distribute`	API Server / Worker	Clipping Worker	Antrean tugas pengunggahan klip ke jaringan platform media sosial.
`clip:boost`	API Server	Clipping Worker	Antrean instruksi automasi promosi interaksi (*engagement boosting*).
`clip:track`	Cron / Worker	Clipping Worker	Antrean pembaruan berkala metriks analitis konten yang telah diunggah.
`clip:dead`	Worker (On Error)	Admin / System Inspector	*Dead-Letter Queue* (DLQ) untuk menampung pesan transaksi yang gagal diproses.

Redis Key-Value & Storage Structures

Pattern Key	Tipe Data	Fungsi & Waktu Simpan (TTL)
`auth:session:<token>`	String / Hash	Menyimpan data otentikasi user aktif. (TTL: 7 Hari / 604800s)
`clip:render:progress:<clipId>`	Hash	Progres status pemrosesan render video (0–100%). (TTL: 1 Jam)
`clipping:worker:heartbeat:<pid>`	String	Sinyal keaktifan *process instance* AI Clipping Worker. (TTL: 30 detik)
`trends:interest:<platform>`	String (JSON)	Cache sementara indeks tren platform terkini. (TTL: 15 Menit)

4. Peta Relasi File Lengkap (Frontend ↔ Backend ↔ DB/Worker)

4.1 Entrypoint & Konfigurasi Inti

```
None
monorepo-root/
├── package.json                          # Workspace root, scripts dev,
concurrently
├── docker-compose.yml                    # PostgreSQL + pgvector (5433) & Redis
(6379)
├── apps/
│   ├── web/                              # React 19 SPA
│   │   ├── src/main.tsx                  # Entrypoint Client, Provider Setup
│   │   ├── src/App.tsx                   # Layout Utama & React Router
Configuration
│   │   └── src/lib/api.ts                # Eden Client (@elysiajs/eden) & SSE
Listener
│   └── api/                              # Elysia Server
│       ├── src/index.ts                  # Entrypoint API Server, SSE
Broadcaster, CORS
│       ├── src/lib/auth.ts               # Session Auth Helper (Redis Store +
Fallback)
│       ├── src/lib/redis.ts              # Client Redis Connection & Subscriber
│       ├── src/db/index.ts               # Drizzle ORM Instance & Postgres Pool
│       ├── src/db/schema.ts              # Skema Drizzle Media Intelligence
│       └── src/db/schema/clipping.ts     # Skema Drizzle AI Video Clipping
├── packages/
│   └── types/                            # @media-intel/types Shared TypeScript
Models
│       └── src/index.ts                  # Export Interface & Data Type
Definitions
└── services/
    └── worker-ai/                        # Python AI Worker
        ├── main.py                       # Multi-process Runner & Entry Point
Worker
        ├── config.py                     # Environment Configuration & Path
Resolvers
        └── db.py                         # Python psycopg Database Driver
Interface
```

4.2 Pemetaan End-to-End per Fitur

A. Autentikasi (`/login`)

- **Frontend View**: `apps/web/src/pages/Login.tsx`
- **API Handler**: `apps/api/src/routes/auth.ts`
- **Core Logic / Session**: `apps/api/src/lib/auth.ts`
- **Database / Cache**: Redis Key (`auth:session:<token>`)

B. Overview & Analytics (`/`)

- **Frontend View**: `apps/web/src/pages/Overview.tsx`
- **API Handler**: `apps/api/src/routes/analytics.ts`
- **Database Source**: PostgreSQL (`media_items`, `topic_clusters`, `alerts`)

C. Media Monitoring & Feeds (`/media`)

- **Frontend View**: `apps/web/src/pages/MediaFeeds.tsx`
- **API Handler**: `apps/api/src/routes/feeds.ts` & `sources.ts`
- **Worker Execution**: `services/worker-ai/scraper.py` & `ingest_worker.py`
- **Database & Stream**: PostgreSQL (`media_items`), Redis Stream (`RAW_STREAM`), Redis Pub/Sub (`media:new`)

D. Topic Clusters (`/clusters`)

- **Frontend View**: `apps/web/src/pages/Clusters.tsx`
- **API Handler**: `apps/api/src/routes/clusters.ts`
- **Worker Execution**: `services/worker-ai/cluster_worker.py` & `nlp.py`
- **Database & Stream**: PostgreSQL (`topic_clusters`, `cluster_items`), Redis Pub/Sub (`clusters:updated`)

E. Trending (`/trending`)

- **Frontend View**: `apps/web/src/pages/Trending.tsx`
- **API Handler**: `apps/api/src/routes/trending.ts`
- **Worker Execution**: `services/worker-ai/trends_poller.py`
- **Database & Cache**: PostgreSQL (`social_trends`), Redis Key (`trends:interest:<platform>`)

F. AI Video Clipping Studio (`/clipping`, `/clipping/:id`)

- **Frontend View**: `apps/web/src/pages/ClippingStudio.tsx` & `ClipDetail.tsx`
- **API Handler**: `apps/api/src/routes/clipping.ts`
- **Worker Execution**:
    - Orchestrator: `services/worker-ai/clipping/worker.py`

    - Pipeline Core: `services/worker-ai/clipping/pipeline.py`
    - Transcriber: `services/worker-ai/clipping/transcriber.py`
    - Curator LLM: `services/worker-ai/clipping/llm_curator.py`
    - Media Renderer: `services/worker-ai/clipping/renderer.py`
    - Auto Publisher: `services/worker-ai/clipping/publisher.py`
- **Database & Stream**: PostgreSQL (`clip_sources`, `generated_clips`, `clip_distributions`), Redis Stream (`clip:ingest`, `clip:render`), Redis Pub/Sub (`clips:updated`)

G. Crisis Alerts & Settings (`/alerts`, `/settings`)

- **Frontend View**: `apps/web/src/pages/Alerts.tsx` & `Settings.tsx`
- **API Handler**: `apps/api/src/routes/alerts.ts` & `keywords.ts`
- **Worker Execution**: `services/worker-ai/alerts_worker.py`
- **Database & Stream**: PostgreSQL (`alerts`, `monitored_keywords`), Redis Pub/Sub (`alerts:new`)

4.3 Komponen UI & Shared Frontend (`apps/web/src/components/`)

- **Layout Application**: `Layout.tsx`, `Sidebar.tsx`, `Header.tsx`
- **Shared Business Components**: `MediaItemCard.tsx`, `ClusterCard.tsx`, `ClipCard.tsx`, `SentimentBadge.tsx`
- **UI Primitives (`/ui`)**: `button.tsx`, `dialog.tsx`, `input.tsx`, `select.tsx`, `badge.tsx`, `card.tsx`
- **Custom Hooks**: `useDebounce.ts`, `useSSE.ts`

4.4 Suite Pengujian (Tests)

- **Backend API Integration Tests**: `apps/api/src/*.test.ts` (Dijalankan menggunakan `bun test`)
- **Python Worker Unit & Pipeline Tests**: `services/worker-ai/tests/` (Dijalankan menggunakan `pytest`)

5. Daftar Lengkap Endpoint API

Sistem Keamanan Auth: Seluruh endpoint bertanda `[Auth]` mewajibkan Header `Authorization: Bearer <token>` atau Session Cookie yang divalidasi oleh middleware `requireAuth`.

Modul / Group	Endpoint	Method	Fungsi & Deskripsi
Root / System	`/`	`GET`	Health status API server.
	`/docs`	`GET`	Dokumentasi Swagger UI interaktif.
	`/api/sse`	`GET`	SSE stream endpoint (`media:new`, `alerts:new`, dll).
Auth	`/api/auth/login`	`POST`	Autentikasi user & pembuatan session token.
	`/api/auth/logout`	`POST`	`[Auth]` Invalidasi session token di Redis.
	`/api/auth/me`	`GET`	`[Auth]` Mengembalikan identitas session user aktif.
Analytics	`/api/analytics/overview`	`GET`	`[Auth]` Ringkasan statistik utama dashboard.
	`/api/analytics/sentimen…`	`GET`	`[Auth]` Metriks statistik volume sentimen harian.
	`/api/analytics/share-of…`	`GET`	`[Auth]` Distribusi proporsi volume sumber media.
Feeds / Media	`/api/feeds`	`GET`	`[Auth]` Daftar artikel berita terfilter dan terpaging.
	`/api/feeds/:id`	`GET`	`[Auth]` Rincian detail item media berita.
Clusters	`/api/clusters`	`GET`	`[Auth]` Daftar kluster isu aktif terkini.
	`/api/clusters/:id`	`GET`	`[Auth]` Detail anggota kluster isu dan riwayat.
Alerts	`/api/alerts`	`GET`	`[Auth]` Daftar peringatan dini krisis.
	`/api/alerts/:id/ack`	`POST`	`[Auth]` Konfirmasi tindakan log peringatan (*acknowledge*).
Keywords	`/api/keywords`	`GET`	`[Auth]` Daftar kata kunci pemantauan sistem.

Modul / Group	Endpoint	Method	Fungsi & Deskripsi
	`/api/keywords`	`POST`	`[Auth]` Mendaftarkan kata kunci pantauan baru.
	`/api/keywords/:id`	`DELETE`	`[Auth]` Menghapus kata kunci pantauan.
Sources	`/api/sources`	`GET`	`[Auth]` Daftar sumber kanal media terdaftar.
	`/api/sources`	`POST`	`[Auth]` Menambahkan sumber scraping baru.
Trending	`/api/trending`	`GET`	`[Auth]` Mengambil tren media sosial terkini.
Clipping Sources	`/api/clipping/sources`	`GET`	`[Auth]` Daftar video sumber ingestion.
	`/api/clipping/sources`	`POST`	`[Auth]` Mendaftarkan URL video YouTube untuk kliping.
	`/api/clipping/sources/:…`	`GET`	`[Auth]` Detail status ingest dan metadata sumber video.
Generated Clips	`/api/clipping/clips`	`GET`	`[Auth]` Daftar seluruh klip hasil prapemrosesan AI.
	`/api/clipping/clips/:id`	`GET`	`[Auth]` Detail status rendering dan transkrip klip.
	`/api/clipping/clips/:id…`	`POST`	`[Auth]` Memicu proses render video klip.
Distributions	`/api/clipping/distribute`	`POST`	`[Auth]` Menjadwalkan/mengunggah klip ke media sosial.
	`/api/clipping/analytics`	`GET`	`[Auth]` Analisis performa tayangan konten video klip.

6. Konfigurasi Environment & Deployment

6.1 Variabel Environment Penting (`.env.example`)

```
None
# Database & Storage
DATABASE_URL="postgresql://postgres:postgres@localhost:5433/media_intel?sslmode
=disable"
REDIS_URL="redis://localhost:6379"

# API & Auth Configuration
PORT=3000
AUTH_USERNAME="admin"
AUTH_PASSWORD="secretpassword"
AUTH_PASSWORD_HASH="$2b$10$YourGeneratedBcryptHashHere..."
JWT_SECRET="super-secret-jwt-token-key"

# AI Core & LLM Models
OPENAI_API_KEY="sk-proj-xxxx..."
GROQ_API_KEY="gsk_xxxx..."
OLLAMA_BASE_URL="http://localhost:11434"
DEFAULT_LLM_PROVIDER="groq" # groq | openai | ollama

# AI Clipping & Media Engine
WHISPER_MODEL="faster-whisper" # faster-whisper | mlx-whisper
WHISPER_MODEL_SIZE="small"     # tiny | base | small | medium | large-v3
FFMPEG_PATH="/usr/bin/ffmpeg"
OUTPUT_DIR="./storage/clips"

# External Web Scraping
TWITTER_BEARER_TOKEN="AAAA..."
YOUTUBE_API_KEY="AIzaSy..."
```

6.2 Docker Compose & Nginx

Sistem dideploy pada lingkungan server Linux menggunakan konfigurasi kontainer terisolasi:

- **`docker-compose.yml`**: Menjalankan PostgreSQL 16 (dengan ekstensi `pgvector`), Redis 7, API Engine (`apps/api/Dockerfile`), serta Python Worker (`services/worker-ai/Dockerfile`).
- **Nginx Proxy (`deploy/nginx/`)**:
  - `ima-isa.conf`: Konfigurasi reverse proxy utama dengan SSL.
  - `ima-isa-http.conf`: Redireksi otomatis HTTP ke HTTPS.

- `ima-isa-headers.conf`: Konfigurasi standar Security Headers (`X-Frame-Options`, `CSP`, HSTS) serta penanganan khusus untuk koneksi persisten SSE (`proxy_set_header Connection ''; proxy_buffering off;`).

7. Panduan Perintah (Menjalankan, Migrasi DB, & Testing)

7.1 Persiapan & Infrastruktur

Jalankan layanan basis data PostgreSQL dan Redis melalui Docker:

```
None
# Menjalankan database PostgreSQL (pgvector) dan Redis secara background
docker compose up -d

# Menginstall seluruh dependensi Monorepo TypeScript (Bun)
bun install

# Menginstall dependensi Python Worker AI (opsional jika dijalankan di host)
cd services/worker-ai
python3 -m venv venv
source venv/bin/activate
pip install -r requirements.txt -r requirements-ai.txt -r
requirements-clipping.txt
cd ../..
```

7.2 Migrasi & Seed Database

```
None
# Menjana file migrasi skema baru jika terdapat perubahan skema Drizzle
bun --filter @media-intel/api db:generate

# Mengeksekusi migrasi skema ke database PostgreSQL
bun --filter @media-intel/api db:migrate

# Mengisi database dengan data awal / dummy untuk pengujian (Seeding)
bun --filter @media-intel/api db:seed
```

7.3 Menjalankan Aplikasi (Development)

```
None
# Menjalankan seluruh stack secara simultan (API, Web, Worker NLP & Worker Clipping)
bun run dev:all

# Menjalankan Backend API dan Frontend SPA saja
bun run dev

# Menjalankan modul Video Clipping Studio secara independen
bun run dev:clip
```

7.4 Build, Typecheck & Testing

```
None
# Memvalidasi tipe TypeScript di seluruh workspace monorepo
bun run typecheck

# Mengeksekusi pengujian otomatis pada Backend API
bun --filter @media-intel/api test

# Mengeksekusi unit test modul Python Worker
cd services/worker-ai && pytest

# Membangun bundle aplikasi produksi untuk Frontend Web dan Backend API
bun run build
```